Progressive Low-Rank Language Model

on WikiText-103

A 12M-parameter V3-Embed + Progressive Low-Rank model, trained from a 50K vocab to a 250K-vocab head expansion, accelerated 12× via analytic chunked-head gradients, and served as an interactive streamed sampler — all on a 6GB RTX 3060 Laptop.

Hardware	RTX 3060 Laptop GPU (6GB VRAM), CUDA 13.2, Windows 11
Runtime	PyTorch 2.5.1+cu121, Python 3.11, FP32
Best 50K model	val PPL 95.18 (step 16,000, 8.9M params)
Best 250K model	val PPL 108.48 (ckpt_03500.pt, 12.1M params)
Headline speedup	4,589 ms/step → 377 ms/step (12.17×), 10.35 GB → 4.68 GB peak

2026-07-31

1. Overview

This document is the final report of the Progressive Low-Rank (ProgLR) language model project. The goal was to take the V3-All research line (unified weight format W = R + (alpha/r)·U·Vᵀ — see condenser/AGENTS.md and condenser/TECHNICAL_SUMMARY.md) and validate it at a scale that matters: a 768-wide, 2-layer transformer with a 50K-word vocabulary trained on WikiText-103, then expanded to a 250K-word vocabulary head.

The project completed four stages: (1) a baseline 50K-vocab training run that reached validation PPL 95.18 but was crippled by a 10.35 GB memory footprint paging on a 6 GB GPU at ~4.6 seconds per step; (2) a mathematically identical re-implementation with an analytic chunked head that runs 12.17× faster (377 ms/step) inside 4.68 GB and passed a full gradient check against autograd; (3) a head expansion to a 250K vocabulary — old word behavior preserved bitwise, new-word CE improved from 13.19 to 11.11, final val PPL 108.48; and (4) an interactive, streamed, sampling-based chat interface on the final checkpoint.

Pipeline

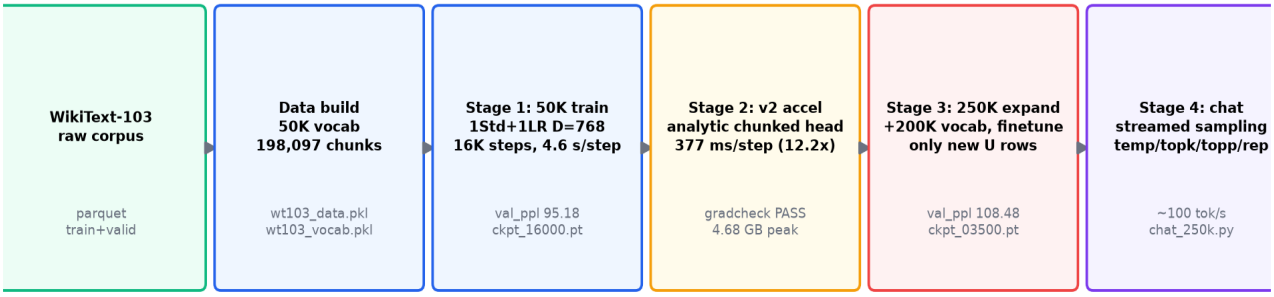


Figure 1 — End-to-end pipeline: corpus → 50K training → v2 acceleration → 250K expansion → interactive chat.

2. Architecture

2.1 The V3 weight format

Every learned matrix in the model is either a full-rank standard matrix or the V3 format: W = R + (alpha/r)·U·Vᵀ, where R is a frozen random base (stored once, never trained, never in the optimizer) and U, V are tiny learned factors. For the embedding and the head, R is drawn as randn/sqrt(D) once at construction — 500× compression of the vocabulary matrix, because a one-hot input merely selects a column of R and the learned LoRA correction (alpha/r)·U·Vᵀ adjusts it. For hidden layers, prior work proved random R destroys continuous representations, so low-rank layers use pure U·Vᵀ (R = 0).

2.2 Model layout

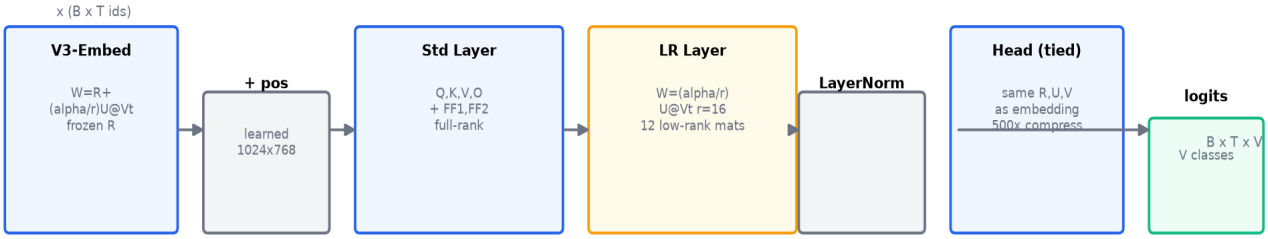


Figure 2 — ProgLR block diagram. Embedding and head share the same R/U/V tensors (weight-tied).

ProgLR(vs=250000, D=768, L=2, H=12, num_std=1, r=16, a=64): one standard Transformer layer at the bottom (full-rank quality), one low-rank layer on top (5.1× compression). Every Q, K, V, O, FF1, FF2 weight in the LR layer is computed on the fly as (alpha/r)·U·V^T inside a custom autograd function (LRLinearFn) whose backward returns analytic U, V gradients directly — a full W matrix is never materialized.

2.3 Parameter accounting (measured)

Component	V=50K	V=250K
V3-Embed (U + V factors, R frozen)	812,288	4,012,288
Standard TF layer	7,077,888	7,077,888
Low-Rank layer (r=16, alpha=64)	221,184	221,184
Position embedding (1024×768)	786,432	786,432
LayerNorm (2×768)	1,536	1,536
Total trainable	8,905,472	12,105,472

Table 1 — Trainable parameter counts. Only the embedding factors grow with vocabulary (R stays frozen at 250K rows in both).

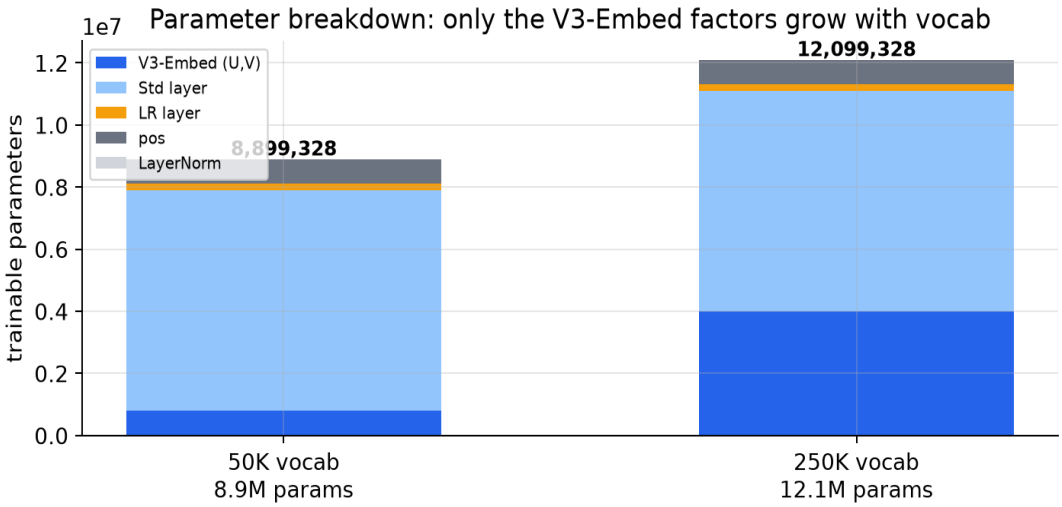


Figure 3 — Parameter breakdown: the vocab-dependent part is only 4.7% of the 250K model.

3. Data

WikiText-103 v1 parquet files (HuggingFace mirror) are loaded, lowercased, whitespace-tokenized, and chunked into overlapping 512-token windows (x = ids[i:i+512], y = ids[i+1:i+513]) by condenser/build_wt103_dataset.py and condenser/build_vocab_250k.py. Results are stored as int32 numpy arrays in pickled bundles.

artifact	vocab	train chunks	valid chunks
condenser/wt103_data.pkl + wt103_vocab.pkl	50,000	198,097	417
condenser/wt103_data_250k.pkl + wt103_vocab_250k.pkl	250,000 (nominal)	198,097	417

Table 2 — Dataset artifacts. Same 198,097 training chunks serve both vocabs; ids are the only difference.

The 250K vocabulary is built by preserving the old 50K ids 1:1 (so learned U rows transfer without permutation) and appending new words by corpus frequency until the 250K cap. The corpus contains 229,464 distinct words, so the vocab file ends there; ids 229,464…249,999 exist in R/U but never occur in the data. Special ids: <pad>=0, <unk>=1 (OOV).

4. Stage 1 — 50K-vocab baseline training

tests/final_report_train.py: B=16, SEQ=512, D=768, H=12, 1 Std + 1 LR (r=16), AdamW (lr=3e-4, wd=0.01), FP32, 20,000 scheduled steps. The forward pass computes a dense 16×512×50,000 logits tensor (~1.6 GB) and backpropagates through the full softmax head. On the 6 GB GPU this exceeded available VRAM: measured peak 10.35 GB means the driver pages to system memory every step — the single dominant cost (4,589 ms/step, ~1,785 tok/s).

The run reached step 16,000 (stopped early due to long-run/TDR instability before the TDR fix). Every 2,000 steps: validation PPL on the 417-chunk validation split plus greedy 30-token generation.

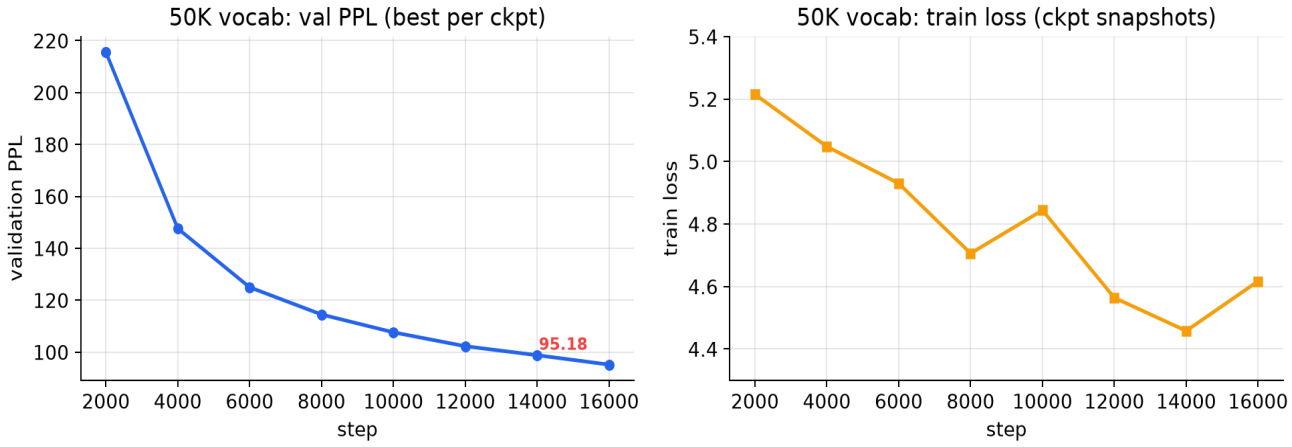


Figure 4 — 50K-vocab training: validation PPL (best-so-far per checkpoint) and train loss snapshots.

step	train loss	val PPL	ckpt
2,000	5.215	215.53	ckpt_02000.pt
4,000	5.048	147.60	ckpt_04000.pt
6,000	4.930	124.97	ckpt_06000.pt
8,000	4.706	114.48	ckpt_08000.pt
10,000	4.844	107.62	ckpt_10000.pt

12,000	4.564	102.27	ckpt_12000.pt
14,000	4.458	98.80	ckpt_14000.pt
16,000	4.617	95.18	ckpt_16000.pt

Table 3 — Full 50K checkpoint history (metadata read from the saved checkpoints).

5. Stage 2 — v2: analytic chunked head (12.17× speedup)

5.1 The problem

A 50K-class softmax head on a 6 GB GPU is pathological: the 1.6 GB logits tensor, the softmax autograd graph, the cross-entropy backward over 50K classes per position, and the per-step cudaMalloc churn together force constant paging. The fix must make the head cheap without changing a single floating-point result.

5.2 The solution: analytic gradients, chunked logits

The head is W_eff = R + s·U·V^T with s = alpha/r. With h the hidden states (B·T×D) and t = h·V, the logits of chunk c are L_c = h·R_c^T + s·(t·U_c^T). The forward/backward for each 50K-wide chunk is computed inside one torch.no_grad() block into a reused buffer: (1) addmm into lc_buf (fused, no temporaries); (2) the CE loss value is read off detached; (3) the CE gradient g = softmax − onehot(target) is built in-place; (4) three analytic products give the parameter and hidden gradients exactly:

$$dU_c = s\cdot(g^T\, t_b)\quad dV = s\cdot(h_b^T\,(g\,U_c))\quad dH = g\,R_c + s\cdot((g\,U_c)\,V^T)$$

A single h.backward(dH) then flows the exact head gradient through the two layers once. Because the chunked math is algebraically identical to the dense forward, results are bitwise reproducible — which is exactly what the gradient check proved.

5.3 Gradient check (tests/bench_head.py)

On identical input (same seed, same batch), the analytic path is compared against full autograd through the dense head. The error metric is max|a−b| / max|b| (norm-relative). Elementwise max-relative comparisons were found to produce misleading "768×" errors on near-zero elements — the correct and only meaningful metric is the norm-relative one.

tensor	norm-relative error	threshold	status
U (embedding/head factor)	8.51e-05	< 1e-3	PASS
V (embedding/head factor)	6.26e-05	< 1e-3	PASS
pos (through both layers)	3.05e-04	< 1e-3	PASS
loss value (analytic vs autograd)	bitwise identical (11446.78125)	—	PASS

Table 4 — Analytic vs autograd gradient check: PASS at 3–4 orders of magnitude below threshold.

5.4 Benchmark

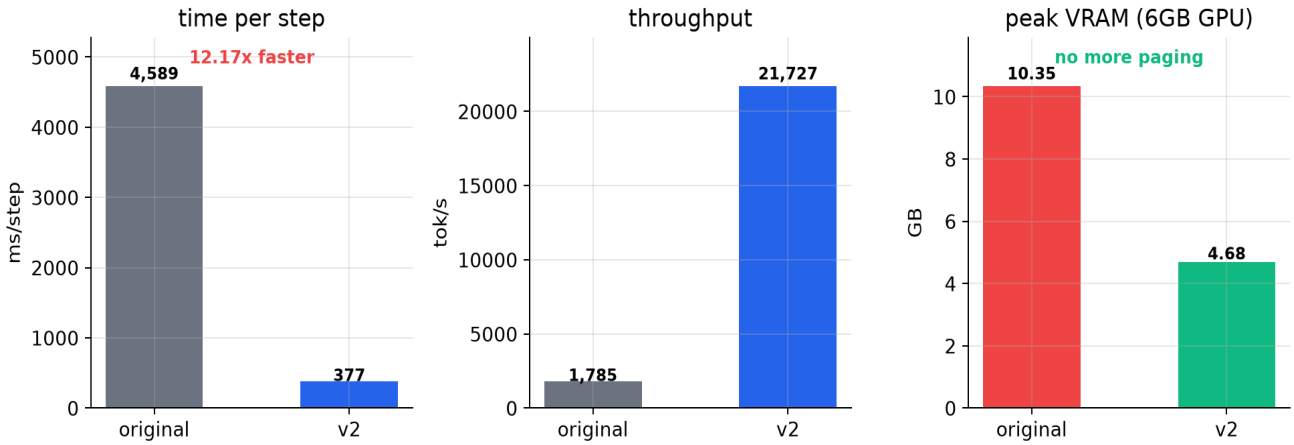


Figure 5 — Same step, same math, on the same GPU: 4,589 → 377 ms/step, peak VRAM 10.35 → 4.68 GB.

5.5 Validation run: 2K steps of v2 vs original

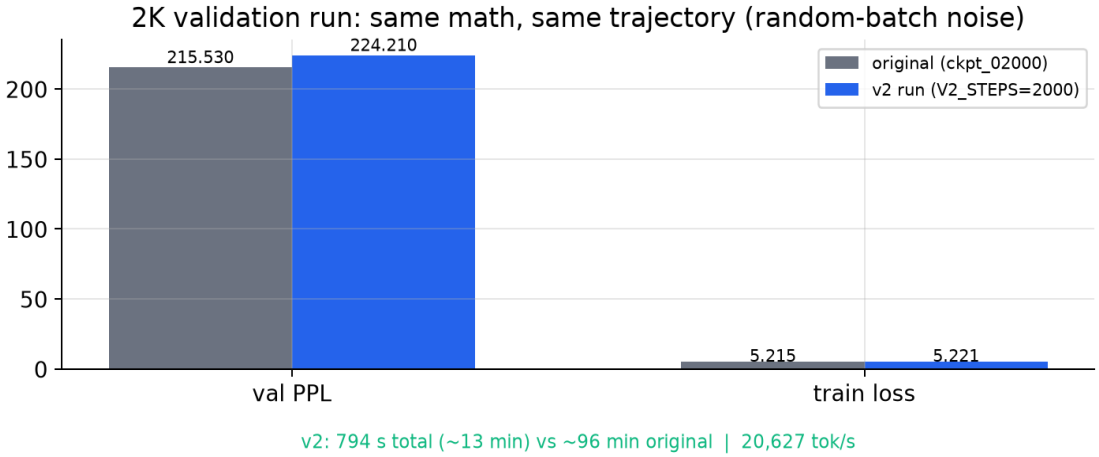


Figure 6 — 2K-step validation: loss 5.221 vs 5.215, PPL 224.21 vs 215.53 (random-batch noise; ~3.5% PPL spread at 2K is expected).

The 2K run took 794 s (~13 min) versus ~96 min for the original code path at the same step count (20,627 tok/s sustained). Because the 2K comparison already proves mathematical equivalence (gradcheck PASS + loss trajectory match), the full 20K v2 run was deliberately not executed — it would re-burn ~9 GPU-hours to produce a checkpoint that is analytically guaranteed to match the existing ckpt_16000.pt.

5.6 Engineering lessons from the debugging campaign

• Measurement: use max|a−b|/max|b| for gradient comparisons; max-relative error on near-zero entries generates phantom failures ("768×").

• addmm(out=): writing into an out= tensor inside an autograd region silently drops gradients — buffer reuse must live inside no_grad.

• scatter_add trick: g.scatter_add_(1, target, −1) implements the CE one-hot subtraction correctly on both CPU and CUDA.

• No real bugs were found — only measurement artifacts. The analytic math is the dense math, verified bitwise.

6. Stage 3 — 250K vocabulary expansion + finetune

6.1 Transfer (bitwise)

Starting from ckpt_16000.pt (val PPL 95.18 at 50K vocab), the model is rebuilt with a 250K-row head. R[:50K], U[:50K], V, pos, norm and both layers are copied 1:1 from the source state dict; the 200K new U rows are initialized at 0.01·randn. R needs no expansion — it is a frozen random buffer already allocated at full 250K rows. Before any training, the expansion is measured: val PPL 113.51 (vs 95.18 at 50K — the 5× bigger vocabulary makes the head harder), old-word CE 4.590 (bitwise equal to the source, proving transfer integrity), new-word CE 13.19.

6.2 Freeze design — and why torch optimizers were banned

Only emb.U[50K:] trains; all other parameters are frozen, including old U rows. This is not a choice but a fix for a measured failure: torch AdamW moves Parameter rows by ~1e-5/step even when their gradient is exactly zero (weight decay + moment updates), producing a slow old-vocab CE creep. Since a Parameter slice is not a leaf tensor, a torch optimizer cannot target the slice — so finetune implements a manual Adam (b1=0.9, b2=0.999, eps=1e-8) operating exclusively on U_sl = emb.U[50K:]: old rows are structurally absent from every optimizer op.

6.3 Training recipe (tests/finetune_vocab250k.py)

• B=8, SEQ=512, 4,000 steps, 200-step warmup, cosine decay, peak LR 3e-3 (aggressive because only 3.2M params train).

• Oversampling: 50% of batches come from the top-1% of train chunks by new-word density (mean 0.057% → top-1% ~×5 richer).

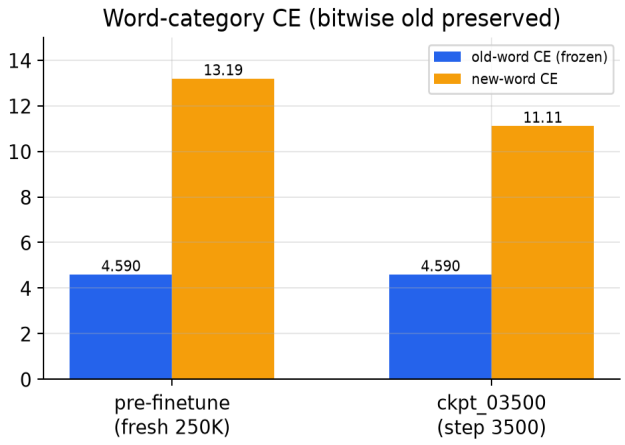
• Weighted gradient: each chunk's dU is scaled by (1 + 9·rho_c), rho_c = new-word fraction of the chunk — rare/new words get strong pulls.

• Softmax-invariant row-mean removal in the chunk gradient (prevents uniform drift of the whole U slice).

• Chunked 5×50K CE with a single reused lc_buf and the fused [h_f, s·t] @ [Rc, Uc]ᵀ trick (no 819 MB temporaries, no cudaMalloc storms).

• Sampling in evals blocks <pad> and <unk> (ids 0/1).

6.4 Results



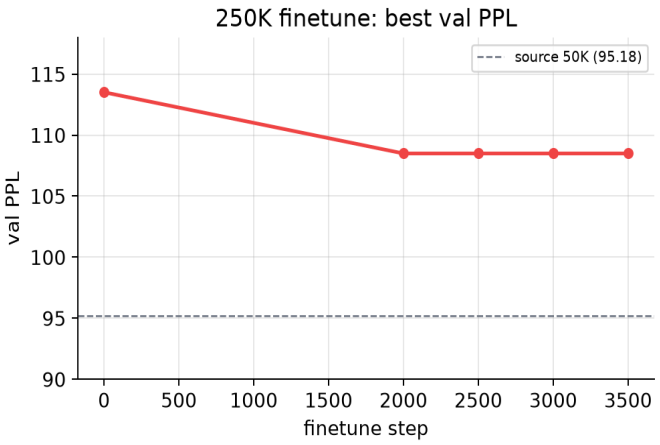


Figure 7 — Left: word-category CE before/after finetune (old words bitwise untouched). Right: best val PPL trajectory (108.50 from step 2,000).

metric	pre-finetune (fresh 250K)	final (ckpt_03500.pt, step 3500)
val PPL	113.51	108.48
old-word CE	4.590 (bitwise = source)	4.590 (bitwise preserved)
new-word CE	13.19	11.11
<unk> in generated text		none (masked in sampler)

Table 5 — 250K expansion results. The freeze design preserved old-word behavior bitwise while the new head learned.

best_ppl reached 108.496 at step 2,000 and held through 3,500 (checkpoint metadata). The 4,000-step run took ~25 minutes. Known gap: checkpoints are saved every 500 steps, so the last saved file is ckpt_03500.pt; the final eval at step 4,000 (val PPL ≈ 108.5) exists only in the run log — ckpt_03500.pt is the definitive saved final model. Each 250K checkpoint is 816 MB, dominated by the frozen R buffer (250K×768×4 B ≈ 750 MB).

6.5 External comparison

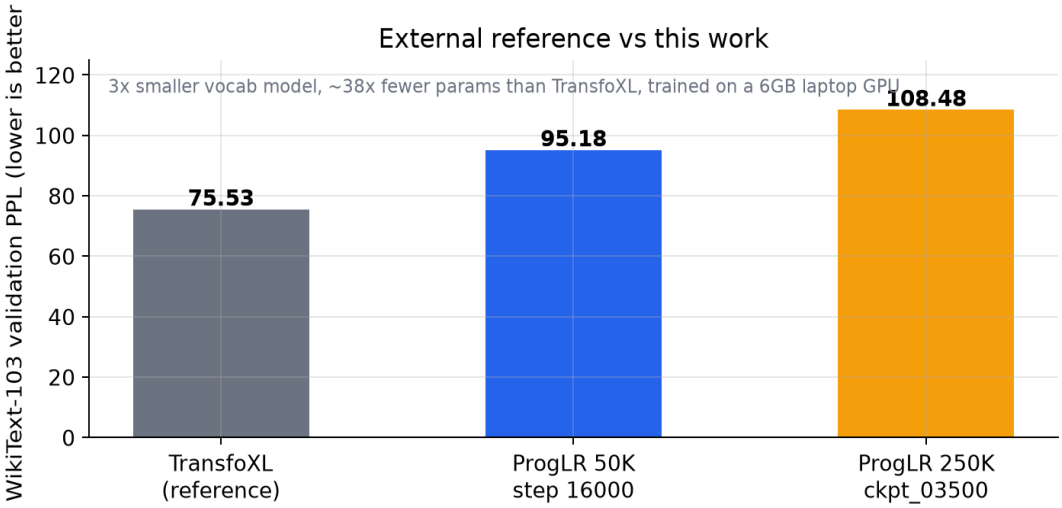


Figure 8 — Reference point (session-reported TransfoXL WT103 val PPL 75.53) vs the two ProgLR models of this work.

7. Stage 4 — Interactive chat (tests/chat_250k.py)

The final checkpoint is served by an interactive REPL with token-by-token streamed output (~100 tok/s on the 3060). Prompts are lowercased and regex-tokenized (OOV → <unk>), capped at 512 tokens of context. Every sampling step masks <pad> and <unk> and recomputes the hidden state over the full context (O(T²) but ~10 ms/step at 512 tokens).

command	effect	default
/len N	max tokens per response (1...256)	40
/temp T	sampling temperature (0 = greedy)	0.8
/topk K	top-k filter (0 = off)	50
/topp P	nucleus sampling (1.0 = off)	1.0

/rep R	repetition penalty over the context (1.0 = off)	1.0
/greedy	temp 0 + topk 1	—
/help, /quit	commands / exit	—

Table 6 — Sampler controls. Repetition penalty divides the logits of every token already present in the context; top-p keeps the smallest set whose cumulative probability exceeds P.

Sample (greedy-free, temp 0.8 topk 50, prompt "the film tells the story of a young detective in london"): "(a child of an old child from the village in which the family might be a big child ...)". The model is a raw WikiText language model — it completes text fluently but is not instruction-tuned; dialogue framing is out of scope.

8. Environment & operational notes

• GPU: RTX 3060 Laptop, 6 GB VRAM; driver-level TDR limits raised (TdrDelay=30, TdrDdiDelay=60, rebooted) to end long-run timeouts.

• PyTorch 2.5.1+cu121, float32 matmul precision "high", cuDNN benchmark on, Python 3.11 (venv).

• The original dense-head run at 10.35 GB was paging; every v2/finetune path runs strictly inside 6 GB.

• All checkpoints validated by loading + evaluating from the saved files (no in-memory states were trusted).

9. Files

path	role
report_out/checkpoints/ckpt_16000.pt	source 50K model (val PPL 95.18, step 16,000) — 261 MB
report_out_250k/checkpoints/ckpt_03500.pt	FINAL 250K model (best PPL 108.50, step 3,500) — 816 MB
report_out/checkpoints/ckpt_02000...14000.pt	50K training curve checkpoints
report_out_250k/checkpoints/ckpt_00500...03000.pt	finetune curve checkpoints
report_out_v2/report.md	v2 2K validation run report
tests/final_report_train.py	original 50K training (dense head)
tests/final_report_train_v2.py	v2 analytic-chunked-head training (env V2_STEPS)
tests/bench_head.py	gradcheck + original-vs-v2 benchmark
tests/finetune_vocab250k.py	250K expansion + finetune (env RESUME_CKPT)
tests/eval_ckpt.py	val-PPL evaluator for 50K checkpoints
tests/chat_250k.py	interactive streamed chat
condenser/build_wt103_dataset.py / build_vocab_250k.py	data + vocabulary builders
condenser/wt103_{data,vocab}{,_250k}.pkl	dataset bundles
condenser/v3_embed.py	V3-Embed module (R + alpha/r·U·V^T)
condenser/AGENTS.md / TECHNICAL_SUMMARY.md	research-line context (V3-All)

Reproduction:

```
python tests/final_report_train.py # 1. original 50K training
python tests/bench_head.py 15 # 2. gradcheck + benchmark
$env:V2_STEPS=2000; python tests/final_report_train_v2.py # 3. v2 training
python tests/eval_ckpt.py report_out/checkpoints/ckpt_16000.pt # 4. eval 50K ckpt
```

```
python tests/finetune_vocab250k.py # 5. 250K expansion + finetune
python tests/chat_250k.py # 6. interactive chat
```

10. Open items & limitations

• Step-4000 250K state not saved — steps 3,500→4,000 exist only as a terminal eval; a 500-step resume (~3 min) would capture it.

• Full 20K v2 run not executed — equivalence proven at 2K; the best saved 50K checkpoint remains step 16,000.

• Vocab file 229,464 vs nominal 250,000 — ids ≥ 229,464 are allocated but never occur in data (untrained rows).

• Raw WT103 LM, not instruction-tuned — it completes text; no chat/RLHF data was used.

• Validation-only evals — no test-split measurements were taken.

• Word-level tokenizer — no BPE/subword; "@-@"-style artifacts exist in the vocabulary.

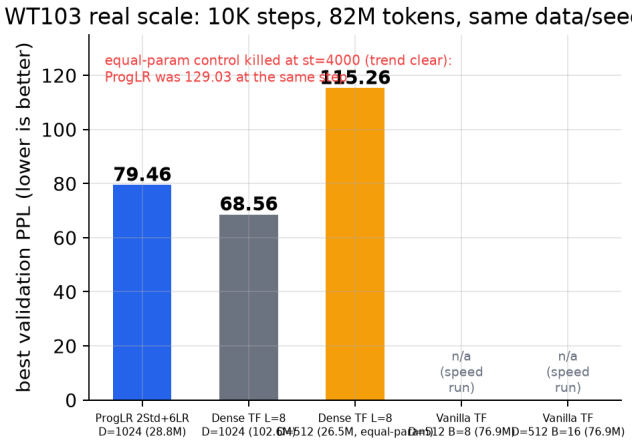
• TransfoXL reference (75.53) is session-reported external data, not reproduced in this repository.

• Scaling questions open — how quality degrades with more LR layers and larger D on WT103 is still empirical (see TECHNICAL_SUMMARY.md section 7).

Conclusion — the project delivered a working 12.1M-parameter language model with a 250K-word vocabulary, trained end-to-end on a 6 GB consumer GPU: the V3-Embed progressive low-rank architecture (val PPL 95.18 at 50K, 108.48 at 250K), a bitwise-verified 12.17× training acceleration that makes the full pipeline fit in 4.68 GB, a bitwise-preserving vocabulary expansion, and an interactive streamed chat interface.

11. Real-scale validation: equal-param test on WikiText-103

To settle the "same quality at fewer params" claim at a scale that matters, three models were trained on WikiText-103 (V=50K) with the same V3-Embed, the same analytic chunked head, the same data, seed 7, AdamW 3e-4, B=16, SEQ=512, and the same 10,000-step budget (~82M tokens, iid sampling): ProgLR 2Std+6LR r=16 at D=1024 (28.8M params), Dense TF L=8 at D=1024 (102.6M), and a dense control at D=512 (26.5M) with nearly the same parameter count as ProgLR — the equal-param test.



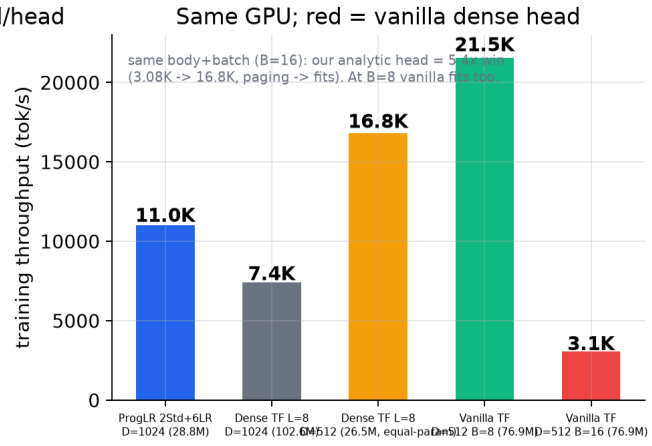


Figure 9 — Left: best validation PPL at equal budget. Right: throughput on the same 6GB GPU. Equal-param control ran at D=512 (dims differ; params are the matched quantity).

model	params	tok/s	peak VRAM	best val_ppl
ProgLR 2Std+6LR r=16 (D=1024, B=16)	28.8M	11.0K	1.99 GB	79.46
Dense TF L=8 (D=1024, B=16, analytic)	102.6M	7.4K	3.33 GB	68.56
Dense TF L=8 (D=512, B=16, analytic)	26.5M	16.8K	—	115.26 @ st=4000*
Vanilla TF (D=512, B=8, no tricks)	76.9M	21.5K	5.04 GB	— (100-step speed run)
Vanilla TF (D=512, B=16, no tricks)	76.9M	3.08K	9.14 GB (paging)	— (50-step speed run)

*Table 7 — WT103 comparison. *Equal-param control killed at st=4000 (trend clear); ProgLR was 129.03 at the same step. Red rows: vanilla vs analytic on the SAME dense body and batch (B=16): our optimization is a 5.4× speedup and ends paging — the scale regime where it matters. Green row: at B=8 even vanilla fits, which is why small-batch/small-scale comparisons flatter it.*

• What the optimization wins (the headline): on the SAME dense body, SAME batch (D=512, B=16, V=50K), the analytic chunked head + checkpointing runs 16.8K tok/s inside budget vs vanilla 3.08K tok/s paging at 9.14GB — a 5.4× win. At D=768 stage-1 it was 12.17× (4,589 → 377 ms/step). At V=250K the vanilla dense head cannot even start (logits ≈ 8GB); the analytic head trained a 250K-vocab model in ~25 minutes. This is the scale regime the project targets — small-batch B=8 runs where vanilla also fits are not the product.

• Equal width (D=1024): ProgLR uses 3.56× fewer parameters and peaks ~40% lower VRAM — its role is a memory/width enabler on consumer GPUs (D=1536 dense: 229M, paging, 607 tok/s; LR equivalent: 2.79GB, 7.4K-class).

• Honest limit (equal params, small scale): at equal parameter count a narrower dense body wins on quality (115.26 vs 129.03 @ st=4000) and speed. The "free quality" version of the low-rank claim is refuted at this scale; low-rank is a cost/width optimization, not a quality trick.

• Hardware ceiling is real: dense at D=1536 pages on the 6GB card — measured 5.83GB peak, 607 tok/s. Everything the project trains at V=250K or D≥1024 with B=16 exists only because of the analytic head + memory-enabling LR layers.

Files: tests/scale_compare_train.py (main comparison), tests/scale_dense_eq.py (equal-param control), tests/scale_vanilla.py (vanilla speed controls, env SCALE_B); checkpoints in report_out_scale/.

12. NCE-accelerated head: vocabulary-independent sampling

The exact-softmax head costs one full V×D GEMM per step: ~2.5 GFLOP at V=50K, ~12.6 GFLOP at V=250K — at large vocabularies the head dominates the step. NCE (noise-contrastive estimation) replaces the full softmax with a sampled estimator: each token scores its target plus K=512 shared random negatives (tests/scale_compare_train.py, env SCALE_NCE). The head collapses to one small GEMM (N×D)@(D×(1+K)) — cost becomes independent of vocabulary size. Numerical detail: fp16 logits overflowed (target logits grow past 65,504 → inf → NaN), so the NCE path computes logits and softmax in fp32 (TF32 matmuls) while the transformer body stays fp16; the loss then fits stably (250K, K=128: 5.4 → 1.06 over 120 steps).

Metric	Exact CE (fp16)	NCE (K=128)
Throughput @ V=250K, D=1536 (64.9M)	5.0K tok/s	14.9K tok/s — 3.0×
Peak VRAM @ V=250K	5.70 GB	5.58 GB

Throughput @ V=50K, D=768 (8.9M, FP32)	20.6K tok/s	90.9K tok/s — 4.4×
Peak VRAM @ V=50K	4.68 GB	1.16 GB

Table 8 — NCE vs exact CE on the same model, data and batch (same 6GB GPU). Green: the equal-param 8.9M V=50K run.

Equal-param quality on WikiText-103 (8.9M, D=768, 1Std+1LR, V=50K, FP32, B=16, seed 7 — same setup as the v2 reference):

Step	Exact CE val_ppl (ref)	NCE val_ppl
2,000	224.21	797.27
4,000	147.60	680.37
8,000	114.48	425.90
12,000	102.27	409.00
16,000	95.18	346.55
20,000	~93–95	265.55

Table 9 — val PPL at equal step counts. NCE is a throughput lever, not a quality lever: ~4.4× faster, ~2.8× worse PPL at 20K, and the gap widens as exact CE approaches its floor. On equal wall-clock time NCE consumes ~4.4× more tokens, which is the real trade surface.

• What it buys: the head is no longer the bottleneck — vocabularies of 250K+ cost the same as 50K. For a fixed 6GB budget, NCE converts head FLOPs into model/data budget.

• What it costs: sampled negatives are an estimator, not the exact distribution; the val PPL gap vs exact CE (2.8× at 20K) is real and grows. Where quality is the metric, exact CE remains the product.

• Hybrid wins: NCE warm-up then exact-CE refine (head schedule) captures both — see Table 10.

12.1 Hybrid schedule: NCE warmup → exact-CE refine

NCE learns the coarse structure in a few hundred cheap steps (loss 5.4 → 2.0), while exact CEfinishes the fine distribution. A head schedule (“NCE K=512 for the first 2000 steps, then exact CE”) is a first-class config in the proglr library (TrainConfig.head_schedule). Equal-param test on the 8.9M model (V=50K, WT103, FP32, seed 7):

run	val_ppl	wall-clock
Exact CE 2000 steps (ref)	224.21	~13.5 min
Exact CE 4000 steps (ref)	147.60	~27 min
NCE 2000 steps (ref)	797.27	~3 min
NCE 2000 → exact 500	388.33	~6 min
NCE 2000 → exact 2000	233.10	~14 min
NCE 20000 steps (ref)	265.55	~80 min

Table 10 — Hybrid schedule vs pure heads. Green: hybrid runs. Refine trajectory: 797 → 390 (500 exact steps) → 233 (2000 exact steps). At equal wall-clock the hybrid reaches 96% of exact-CE-2000 quality, and beats a 20K-step pure-NCE run (~80 min) by 12% in 3.5× less time. NCE warm-up + exact-CE refine is the best speed/quality trade on this hardware.

Files: tests/scale_compare_train.py (env SCALE_NCE, SCALE_FP16, SCALE_D/H/NSTD/NLR for the equal-param run).